

Resumen Teórico: Complejidad Temporal

Computabilidad y Complejidad — UNRC

Basado en el material de Pablo Castro (UNRC)

Índice

1. Introducción	1
2. Tiempo de ejecución como función	2
3. Notaciones asintóticas	3
3.1. Notación O (big-oh): cota superior	3
3.2. Notación o (little-oh): cota superior estricta	3
3.3. Notación Ω : cota inferior	3
3.4. Notación Θ : igualdad de orden	3
4. Análisis de tiempo de ejecución: un ejemplo	4
4.1. Primera solución: $O(n^2)$	4
4.2. Una solución mejor: $O(n \log n)$	4
5. Clases de complejidad	5
6. La clase P: problemas tratables	5
6.1. Comparación de órdenes de crecimiento	6
6.2. Definición de la clase P	6
7. Modelos de computación y simulación polinomial	6
7.1. Simulación: multicinta $\Rightarrow$ una sola cinta	6
7.2. Tesis de Church-Turing Extendida	7
8. Tiempo en máquinas de Turing no deterministas	7
8.1. Definición	7
8.2. Simulación: no determinista $\Rightarrow$ determinista	8
9. Ejemplos de problemas en P	8
9.1. Conectividad en grafos: <i>Path</i>	8
9.2. Coprimos: aplicación del algoritmo de Euclides	8
9.3. Pertenencia para gramáticas libres de contexto (CYK)	9
10. Resumen final	10

1. Introducción

Hasta ahora vimos qué problemas se pueden *computar* y cuáles no. El estudio de la **computabilidad** responde a la pregunta:

Dado un problema P , ¿es computable?

La **complejidad computacional** cambia el enfoque: ya trabajamos solo con problemas *computables* y nos preguntamos:

*¿Hay algoritmos **eficientes** para resolver P ?*

Este resumen recorre las herramientas básicas para responder esta pregunta:

1. Cómo medir el tiempo de ejecución de un algoritmo.
2. Las notaciones asintóticas O, o, Ω, Θ .
3. Las clases de complejidad $\text{Time}(t(n))$ y, en particular, la clase P .
4. Cómo se relacionan los distintos modelos de computación bajo la lupa temporal: MTs con una cinta, multicinta y no determinista.
5. La *Tesis de Church-Turing extendida*: todos los modelos razonables son equivalentes en tiempo *módulo polinomios*.
6. Ejemplos clásicos de problemas en P : caminos en grafos, números coprimos y pertenencia a gramáticas libres de contexto.

Intuición

Una pregunta importante antes de hablar de eficiencia: *¿qué modelo de computación usamos como referencia?* La respuesta es: las máquinas de Turing deterministas. Veremos que los modelos razonables se simulan entre sí con un costo *a lo sumo polinomial*, así que la noción de “algoritmo eficiente” es robusta.

2. Tiempo de ejecución como función

Dada una máquina de Turing M que termina en todas sus entradas, su **tiempo de ejecución** es una función

$$T_M : \mathbb{N} \rightarrow \mathbb{R}^{\geq 0},$$

donde $T_M(n)$ representa la cantidad máxima de pasos que realiza M para una entrada de tamaño n (análisis del **peor caso**).

Convenciones habituales:

- T_M es **monótona** (no decrece con n).
- Trabajamos con MTs que terminan siempre.
- Se analiza típicamente el **peor caso**, no el caso promedio.
- Nos interesa el *comportamiento asintótico*: cómo crece T_M cuando n se hace grande.

Intuición

No interesa el valor exacto de $T_M(7)$, sino cómo escala $T_M(n)$ cuando $n \rightarrow \infty$. Las constantes multiplicativas y los términos de menor orden no son lo importante: lo que importa es el *orden de crecimiento*.

3. Notaciones asintóticas

Para hablar del orden de crecimiento de las funciones se usan varias notaciones, todas ellas basadas en comparaciones asintóticas.

3.1. Notación O (big-oh): cota superior

Definición 3.1 (Big-oh). $f(n) \in O(g(n))$ si y solo si existen constantes $c > 0$ y $n_0 > 0$ tales que

$$\forall n \geq n_0 : f(n) \leq c \cdot g(n).$$

Intuición: para entradas suficientemente grandes, g crece *al menos tan rápido* como f , salvo una constante multiplicativa.

Ejemplo 3.2. $3n^2 + 5n + 100 \in O(n^2)$. Tomando $c = 4$ y n_0 adecuado, $3n^2 + 5n + 100 \leq 4n^2$ a partir de cierto punto.

3.2. Notación o (little-oh): cota superior estricta

Definición 3.3 (Little-oh). $f(n) \in o(g(n))$ si y solo si para *toda* constante $c > 0$ existe $n_0 > 0$ tal que

$$\forall n \geq n_0 : f(n) < c \cdot g(n).$$

Una caracterización equivalente, muy útil, es mediante un límite:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \iff f(n) \in o(g(n)).$$

Intuición

o es una versión *estricta* de O : la función g crece *estrictamente* más rápido que f . En particular, $f(n) \notin o(f(n))$ (uno no crece estrictamente más rápido que sí mismo), aunque sí $f(n) \in O(f(n))$.

Ejemplo 3.4. $n \in o(n^2)$, $\log n \in o(n)$, $n \in o(n \log n)$.

3.3. Notación Ω : cota inferior

Definición 3.5 (Omega). $f(n) \in \Omega(g(n))$ si y solo si existen constantes $c > 0$ y $n_0 > 0$ tales que

$$\forall n \geq n_0 : f(n) \geq c \cdot g(n).$$

Intuición: para entradas grandes, f crece *al menos tan rápido* como g (es la cota *inferior*, opuesta a O).

3.4. Notación Θ : igualdad de orden

Definición 3.6 (Theta). $f(n) \in \Theta(g(n))$ si y solo si $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$.

Intuición: f y g crecen “al mismo ritmo” — una está sándwich entre dos múltiplos constantes de la otra.

Ejemplo 3.7. $3n^2 + 5n + 100 \in \Theta(n^2)$.

Idea clave

Resumen de las cuatro notaciones:

- $O(g)$: “ f no crece más rápido que g ”.
- $o(g)$: “ f crece *estrictamente* más lento que g ”.
- $\Omega(g)$: “ f no crece más lento que g ”.
- $\Theta(g)$: “ f crece exactamente igual que g ”.

4. Análisis de tiempo de ejecución: un ejemplo

Estudiemos cuánto tarda una MT en decidir el lenguaje clásico

$$A = \{0^k 1^k \mid k \geq 0\}.$$

4.1. Primera solución: $O(n^2)$

Algoritmo (alto nivel). Dado el input w :

1. Recorrer la cinta; si se encuentra “10”, rechazar.
2. Volver al principio. Tachar el primer 0, buscar el primer 1 y tacharlo.
3. Repetir el paso 2 hasta que:
 - No queden 0s ni 1s: *aceptar*.
 - Queden solo 0s o solo 1s: *rechazar*.

Análisis paso a paso para una entrada de longitud n :

- El paso 1 (verificar formato 0^*1^*): $O(n)$ pasos.
- El paso 2 (tachar un par 0/1): cada iteración requiere recorrer la cinta, es decir $O(n)$ pasos.
- El paso 2 se repite hasta $n/2$ veces, una por cada par a tachar.

Tiempo total:

$$O(n) + O\left(\frac{n}{2} \cdot n\right) = O(n) + O(n^2) = O(n^2).$$

Conclusión: esta MT decide A en tiempo $O(n^2)$.

4.2. Una solución mejor: $O(n \log n)$

Idea: en cada pasada, eliminar la *mitad* de los 0s y la mitad de los 1s. Si en alguna pasada la cantidad de 0s (o 1s) es impar y mayor a 1, rechazar.

Algoritmo.

1. Recorrer la cinta; si aparece “10”, rechazar.
2. Repetir mientras queden 0s o 1s:
 - Contar la paridad de 0s y 1s. Si alguno es impar (y mayor que 1), rechazar.
 - Tachar uno de cada dos 0s alternadamente, y lo mismo con los 1s.
3. Si al terminar no quedan 0s ni 1s, aceptar; si no, rechazar.

Análisis.

- El paso 1: $O(n)$.
- El bucle del paso 2 se ejecuta $O(\log n)$ veces, porque en cada iteración *se divide la cantidad por 2*.
- Cada iteración del bucle realiza $O(n)$ pasos (contar paridad y tachar alternadamente).

Tiempo total:

$$O(n) + O(\log n) \cdot O(n) = O(n \log n).$$

Intuición

La mejora ($O(n^2) \rightarrow O(n \log n)$) se obtiene aprovechando que cada pasada *divide a la mitad* el trabajo restante, igual que en *binary search*. La idea es: “cada vez hago la mitad” produce un factor $\log n$ donde antes había un n .

Ejemplo 4.1 (Trazado para $n = 50$, $w = 0^{25}1^{25}$).
▪ Pasada 1: paridad OK; tacho 12 ceros y 12 unos. Quedan 13 ceros y 13 unos. Total tachado: 24+24.

- Pasada 2: paridad OK; tacho 6 ceros y 6 unos.
- Pasada 3: tacho 3 y 3.
- Pasada 4: tacho 2 y 2.
- Pasada 5: tacho 1 y 1. Quedan 0s. Aceptar.

Cantidad de pasadas: $\Theta(\log n)$.

5. Clases de complejidad

Habiendo definido cómo medir el tiempo, podemos *clasificar* los problemas según el orden de crecimiento del algoritmo más rápido que los resuelve.

Definición 5.1 (Clase Time).

$$\text{Time}(t(n)) = \{L \mid L \text{ se puede decidir con una MT en } O(t(n)) \text{ pasos}\}.$$

Ejemplo 5.2. $\text{Time}(O(n^2))$ contiene a todos los problemas que se pueden resolver con un algoritmo que en el peor caso hace $c \cdot n^2$ pasos para alguna constante c . Sorting es un ejemplo clásico.

Preguntas centrales que estudia la complejidad:

- Dado un problema, ¿pertenece a $\text{Time}(O(t(n)))$ para algún t específico?
- ¿Hay problemas en $\text{Time}(O(t(n)))$ pero no en $\text{Time}(O(t'(n)))$ para $t' < t$?

6. La clase P : problemas tratables

Los algoritmos que corren en tiempo *polinomial* reciben un nombre especial: se dicen **tratables**. La razón es eminentemente práctica.

6.1. Comparación de órdenes de crecimiento

La siguiente tabla, asumiendo 10^6 operaciones por segundo, muestra cuán dramática es la diferencia entre los órdenes polinomial y exponencial:

	$n = 10$	$n = 20$	$n = 30$	$n = 40$	$n = 50$
n	0.00001 s	0.00002 s	0.00003 s	0.00004 s	0.00005 s
n^2	0.0002 s	0.0004 s	0.0009 s	0.0016 s	0.0025 s
n^3	0.1 s	3.2 s	24.3 s	1.7 min	15.2 min
2^n	0.001 s	1.0 s	17.9 min	12.7 días	35.6 años

Intuición

Los algoritmos polinomiales escalan bien: pasar de $n = 10$ a $n = 50$ multiplica el tiempo por algo manejable. Los algoritmos exponenciales colapsan: un problema que se resuelve en un segundo para $n = 20$ tarda *décadas* para $n = 50$. Por eso identificamos “tratable” con “polinomial” a nivel teórico.

6.2. Definición de la clase P

Definición 6.1 (Clase P).

$$P = \bigcup_{k>0} \text{Time}(O(n^k)).$$

Es decir, P es la clase de los problemas para los cuales *existe algún k* y un algoritmo polinomial de grado k que los decide.

Propiedades:

- P es **invariante** respecto a los modelos de computación deterministas razonables (Tesis de Church-Turing Extendida).
- Se considera la clase de los problemas *eficientes*: un avance en el hardware (multiplicar la velocidad por un factor constante) tiene impacto real sobre problemas polinomiales, pero apenas mueve la aguja para problemas exponenciales.

7. Modelos de computación y simulación polinomial

Una pregunta importante: ¿depende la clase P del modelo de computación elegido?

Hay varios modelos relevantes:

- MTs deterministas con una sola cinta.
- MTs deterministas con varias cintas.
- MTs deterministas con cinta doblemente infinita.
- MTs no deterministas.

Resultado clave: los *tres primeros* (todos los modelos deterministas razonables) se simulan entre sí en *tiempo polinomial*. Por lo tanto, definen la *misma* clase P .

7.1. Simulación: multicinta $\Rightarrow$ una sola cinta

Teorema 7.1. Si una MT multicinta M decide un problema en $O(t(n))$ pasos, entonces existe una MT con una sola cinta S que lo decide en $O(t(n)^2)$ pasos.

Idea de la simulación.

- En la cinta única de S se concatenan las k cintas de M , separadas por un símbolo especial $\#$.
- Para cada cinta, se usa una marca sobre un símbolo (por ejemplo, $\bar{a}$) para indicar la posición del cabezal.
- Para simular *un paso* de M : S recorre todas sus celdas para localizar las k cabezas marcadas, aplica la transición y vuelve a recorrer para actualizar los símbolos y mover los cabezales virtuales.

Análisis del costo.

- Por cada paso de M , los contenidos virtuales tienen a lo sumo longitud total $O(t(n))$ (porque cada cinta no puede crecer más rápido que el tiempo gastado).
- Simular un paso de M cuesta $O(t(n))$ pasos a S .
- Como M hace $O(t(n))$ pasos en total, S hace $O(t(n)) \cdot O(t(n)) = O(t(n)^2)$ pasos.

Idea clave

El factor cuadrático es *polinomial*: si $t(n)$ era un polinomio, $t(n)^2$ también lo es. Por lo tanto, la clase P no cambia al pasar de MTs multicinta a una sola cinta.

7.2. Tesis de Church-Turing Extendida

Tesis de Church-Turing Extendida

Si un problema es decidable en tiempo $O(t(n))$ por *algún* modelo razonable de computación **determinista**, entonces es decidable en $O(t(n)^k)$ por *cualquier* otro modelo razonable determinista, para algún k .

Consecuencia: la clase P es *robusta* en sentido fuerte. No depende de si elegimos máquinas de Turing, máquinas RAM, lambda cálculo, o un lenguaje de programación moderno: el conjunto de problemas resolubles en tiempo polinomial es el mismo.

Observación 7.2. La tesis tiene la palabra **determinista** subrayada. El caso *no determinista* es distinto: la simulación de una MT no determinista por una determinista cuesta *exponencial*, no polinomial. Esta brecha entre determinismo y no determinismo es la raíz del famoso problema P vs. NP (que se aborda en los teóricos posteriores).

8. Tiempo en máquinas de Turing no deterministas

8.1. Definición

Las máquinas de Turing no deterministas pueden tener varios sucesores en cada configuración. Por eso, la ejecución sobre una entrada forma un *árbol*, no una secuencia lineal.

Definición 8.1 (Tiempo en MT no determinista). El tiempo de ejecución de una MT no determinista N sobre una entrada de tamaño n es el largo de la rama *más larga* del árbol de cómputo. Es decir, $T_N(n) \leq t(n)$ significa que *todas* las ramas posibles tienen a lo sumo $t(n)$ pasos.

8.2. Simulación: no determinista $\Rightarrow$ determinista

Teorema 8.2. Si una MT no determinista N decide un problema en $O(t(n))$ pasos, entonces existe una MT determinista M que lo decide en $2^{O(t(n))}$ pasos.

Idea (BFS sobre el árbol de ejecución).

- Sea b el factor de ramificación máximo (cantidad máxima de sucesores de una configuración). Como $\delta(q, a)$ es un conjunto finito, b está acotado por una constante.
- El árbol de ejecución tiene profundidad $\leq t(n)$, así que su cantidad total de nodos es a lo sumo $b^{t(n)}$.
- Si $b = 2^c$ para alguna constante c , entonces $b^{t(n)} = 2^{c \cdot t(n)} \in 2^{O(t(n))}$.

La máquina determinista M recorre todo el árbol con BFS y acepta si y solo si alguna rama acepta. Como debe visitar a lo sumo todos los nodos del árbol, el costo es $2^{O(t(n))}$.

Intuición

La simulación *no es polinomial*: pasar de tiempo $O(t(n))$ no determinista a determinista cuesta una explosión exponencial. Este es el motivo por el cual P y NP podrían ser clases distintas; la pregunta abierta es si esta simulación es óptima o si existe alguna técnica más inteligente que evite la explosión.

9. Ejemplos de problemas en P

9.1. Conectividad en grafos: *Path*

Ejemplo 9.1. Consideremos

$$Path = \{ \langle G, v, w \rangle \mid G \text{ es un grafo con un camino de } v \text{ a } w \}.$$

Algoritmo ineficiente. Generar todos los caminos de longitud 1, 2, 3, ..., hasta $|V|$, y verificar si alguno conecta v con w . La cantidad de posibles caminos es $|V|!$, así que el algoritmo es $O(n!)$. *No tratable*.

Algoritmo eficiente (BFS / marcado).

1. Marcar v .
2. Mientras se agreguen nuevos nodos marcados:
 - Recorrer las aristas; para cada arista (a, b) con a marcado y b no, marcar b .
3. Si w resultó marcado, aceptar; si no, rechazar.

Análisis. El bucle se ejecuta a lo sumo $|V|$ veces (cada iteración marca al menos un nodo nuevo), y cada pasada recorre las aristas en $O(|E|)$. Total: $O(|V| \cdot |E|) \in O(n^2)$. **Polinomial**, así que $Path \in P$.

9.2. Coprimos: aplicación del algoritmo de Euclides

Ejemplo 9.2.

$$Coprimos = \{ \langle x, y \rangle \mid \gcd(x, y) = 1 \}.$$

Algoritmo ineficiente (fuerza bruta). Para cada $z \leq \min(x, y)$, verificar si z divide a ambos. Como los números están codificados en *binario* con n bits, hay 2^n valores de z que probar. Tiempo 2^n : *exponencial* en el tamaño de la entrada. **No es polinomial.**

Algoritmo eficiente (Euclides).

1. Repetir hasta que $y = 0$:
 - $x \leftarrow x \bmod y$.
 - Intercambiar x y y .
2. Si $x = 1$, aceptar (son coprimos); si no, rechazar.

Análisis.

- En cada iteración, x se reduce *al menos a la mitad*.
- Si los números tienen n bits, alcanzan $O(n)$ iteraciones para terminar.
- Cada operación módulo se computa en tiempo polinomial $p(n)$.

Total: $O(n \cdot p(n))$, **polinomial**.

Idea clave

Atención al tamaño de la entrada: para problemas numéricos, “tamaño de la entrada” significa *cantidad de bits*, no el valor del número. Un algoritmo “en x pasos” con x del orden de 2^n *no* es polinomial. Por eso la fuerza bruta sobre divisores no califica.

9.3. Pertenencia para gramáticas libres de contexto (CYK)

Ejemplo 9.3.

$$GLC = \{\langle G, w \rangle \mid w \text{ es generado por la gramática libre de contexto } G\}.$$

Idea. Usar *programación dinámica* y la **Forma Normal de Chomsky**, donde las producciones son $A \rightarrow BC$ o $A \rightarrow a$ (o $A \rightarrow \varepsilon$ para la cadena vacía).

Estructura de datos. Dado $w = w_1 w_2 \cdots w_n$, construimos una matriz $M[i][j]$, donde $M[i][j]$ es el conjunto de variables que pueden generar la subcadena $w_i \cdots w_j$.

Algoritmo (CYK).

1. Si $w = \varepsilon$: aceptar si $S \rightarrow \varepsilon$ es una regla.
2. Para $i = 1, \dots, n$ y cada variable A :
 - Si $A \rightarrow w_i$ es regla, agregar A a $M[i][i]$.
3. Para $l = 2, \dots, n$ (longitud de la subcadena):
 - Para $i = 1, \dots, n - l + 1$ y $j = i + l - 1$:
 - Para $k = i, \dots, j - 1$ y cada regla $A \rightarrow BC$:
 - Si $B \in M[i][k]$ y $C \in M[k + 1][j]$, agregar A a $M[i][j]$.
4. Aceptar si $S \in M[1][n]$.

Análisis.

- Los bucles anidados sobre l, i, k y reglas hacen $O(|V| \cdot n^3)$ trabajo, donde $|V|$ es la cantidad de variables. Es **polinomial**.

Por lo tanto $GLC \in P$.

Intuición

El algoritmo CYK es un caso bonito de *programación dinámica*: en lugar de explorar todas las derivaciones (lo que sería exponencial), reutilizamos resultados anteriores. La forma normal de Chomsky es la herramienta que permite “trocear” las derivaciones en sub-derivaciones más pequeñas.

10. Resumen final

Ideas centrales para llevar:

1. **Complejidad vs. computabilidad.** Computabilidad pregunta “¿se puede?”; complejidad pregunta “¿se puede rápido?”. En complejidad ya trabajamos con problemas computables.
2. **Tiempo como función.** $T_M : \mathbb{N} \rightarrow \mathbb{R}^{\geq 0}$ mide el peor caso. Lo que interesa es el comportamiento asintótico, no las constantes.
3. **Notaciones asintóticas.**
 - $O(g)$: cota superior. $f \leq c \cdot g$ asintóticamente.
 - $o(g)$: cota superior estricta. $\lim f/g = 0$.
 - $\Omega(g)$: cota inferior.
 - $\Theta(g)$: mismo orden de crecimiento (ambas).
4. **Análisis cuidadoso paga.** El mismo problema $\{0^n 1^n\}$ se resuelve en $O(n^2)$ con un algoritmo ingenuo y en $O(n \log n)$ con una idea de “dividir a la mitad”.
5. **Clases de complejidad.** $\text{Time}(t(n))$ agrupa los problemas decidibles en $O(t(n))$ pasos por una MT determinista.
6. **La clase P .** $P = \bigcup_{k \geq 0} \text{Time}(O(n^k))$: los problemas resolubles en tiempo polinomial. Es la formalización de “problemas tratables”. Los polinomios escalan bien; los exponenciales no.
7. **Simulación polinomial entre modelos deterministas.** MTs multicinta se simulan en una cinta con costo $O(t(n)^2)$. Esto es polinomial, así que P no depende del modelo (determinista) elegido.
8. **Tesis de Church-Turing Extendida.** Todos los modelos razonables *deterministas* se simulan entre sí en tiempo polinomial. Por lo tanto, la noción de “algoritmo polinomial” es invariante de modelo.
9. **El no determinismo cambia las reglas.** Una MT no determinista de tiempo $t(n)$ se simula con una determinista de tiempo $2^{O(t(n))}$: explosión exponencial. Esta brecha motiva la clase NP y el famoso problema $P \stackrel{?}{=} NP$.
10. **Ejemplos de P .** *Path* (BFS, $O(|V| \cdot |E|)$), *Coprimos* (Euclides, $O(n \cdot p(n))$), *GLC* (CYK, $O(|V| \cdot n^3)$). **Importante:** el tamaño de la entrada en problemas numéricos es la cantidad de *bits*, no el valor del número.

Intuición

La clase P es el punto de partida de toda la complejidad computacional. Definir “eficiente” como “polinomial” no es perfecto (un algoritmo $O(n^{100})$ es polinomial pero impráctico), pero captura mucho mejor que cualquier otra alternativa la intuición de que “los buenos algoritmos escalan”. A partir de P se construye la jerarquía de clases (NP , PSPACE, etc.), las nociones de *reducción polinomial* y, eventualmente, las preguntas que aún hoy permanecen abiertas en la teoría de la computación.